PA3 Overview Document

CS 4120: Overview Documentation

Programming Assignment 3: Semantic Analysis (Type Checking)

Authors: Fadi Ismail (fmi4), Yoseph Endawoke (ye38), Charles Liggins (cll258), Temi Adebowale (ta375)

Date: February 28, 2026

---

1. Metadata

- **Assignment:** Programming Assignment 3 — Type Checker
- **Group members:**
  - Fadi Ismail (fmi4)
  - Yoseph Endawoke (ye38)
  - Charles Liggins (cll258)
  - Temi Adebowale (ta375)
- **How to run:**

```
# Build the compiler
./etac-build

# Type check a single .eta file
./etac --typecheck <filename>.eta

# Type check with library path and output directory
./etac --typecheck -libpath <libdir> -D <outdir> --sourcepath <srcdir> <files...>

# Lex and parse still work (PA1/PA2 functionality preserved)
./etac --lex <filename>.eta
./etac --parse <filename>.eta

# Run unit tests
cargo test
```

- **Main entry point:** `src/main.rs`
- **Environment tested:** macOS (Darwin 25.2.0), Linux VM (Ubuntu 24.04, course Docker image), Rust 1.93.0

---

2. Summary

The most challenging aspects of this assignment were designing a clean error handling strategy using Rust's `Result` type and propagating source location information through the AST for accurate error reporting.

Key design decisions included:

- **Fail-fast error handling with `Result` types:** Rather than accumulating errors in a list, the type checker returns `Result<SemanticType, SemanticError>` from every checking function and uses the `?` operator for immediate propagation. This simplified control flow and made error handling uniform across all checking logic.

- **`Spanned<T>` wrapper for AST span propagation:** Instead of adding span fields to every AST node, we introduced a generic `Spanned<T>` struct that pairs any node with a `Span { start, end }` byte-

offset range. The type aliases `Expr = Spanned<ExprKind>` and `Stmt = Spanned<StmtKind>` make this transparent to consumers while providing location data for every expression and statement.

- **Scoped symbol table with mutation log:** The type context uses a `HashMap`-based symbol table with a mutation log and scope markers for efficient `enter_scope`/`exit_scope` operations, supporting correct variable shadowing across nested scopes.
- **Two-pass type checking:** The checker first collects all top-level function and global signatures into the context, then checks each function body. This allows forward references (functions calling other functions declared later in the file).
- **Interface registration:** Parsed `.eti` interface files are loaded and their function signatures are registered in the type context before program checking begins, enabling use of library functions like those in `io.eti`.

A known limitation is that the type checker currently uses `context.clone()` to save and restore scope state around blocks, if/else, and while statements. This works correctly but is less efficient than the `enter_scope`/`exit_scope` mechanism provided by the scoped symbol table.

---

3. Specification

We encountered several areas requiring interpretation:

- **Error reporting format:** The specification requires `<line>:<col> error:<description>` output. We capture byte offsets via LALRPOP's `@L`/`@R` markers in the grammar and convert them to 1-indexed line and column numbers using a `byte_offset_to_line_col` utility. For valid programs, the output is `Valid Eta Program`.
- **Fail-fast vs. multi-error:** The specification does not prescribe whether the compiler should report all errors or stop at the first. We chose fail-fast (report the first error encountered and stop), which is simpler and consistent with most production compilers' behavior for semantic errors.
- **`void` vs. `unit` statement types:** We distinguish between `Void` (a statement that always returns, e.g., a block ending in `return`) and `Unit` (a statement that may or may not return). An `if/else` where both branches are `Void` produces `Void`; otherwise it produces `Unit`. A function with a non-empty return type must have a `Void` body.
- **Empty array literal typing:** The expression `{}` has type `EmptyArray`, which is compatible with any array type. This allows `x: int[] = {}` without knowing the element type at the literal site.
- **Procedure call representation:** A procedure call `f(x)` is represented as `Assign([], [FuncCall("f", [x])])` — an assignment with no targets and one expression. The type checker special-cases this pattern to verify argument types without requiring a return value.
- **Multi-return assignment:** `x, y = f()` is checked by verifying that the function's return tuple length matches the number of assignment targets, and that each target's type matches the corresponding return type. Discard targets (_) accept any type.
- **Single-dash CLI flags:** The `eth` test harness passes `-libpath` and `-sourcepath` (single-dash, Java-style), but our CLI uses `clap` which expects `--libpath`/`--sourcepath` (double-dash). We normalize these in a `normalize_args()` preprocessing step before parsing.

---

4. Design and Implementation

4.1 Architecture

Module layout (new and modified modules for PA3):

Module	File	Responsibility
Type Checker	`src/checker/check.rs`	Core type checking logic: expressions, statements, functions, programs
Context	`src/checker/context.rs`	Scoped symbol table (`ScopedSymbolTable<V>`) and `Context` wrapper
Context Tests	`src/checker/context_tests.rs`	Unit tests for the scoped symbol table
AST (modified)	`src/parser/ast.rs`	Added `Span`, `Spanned<T>`, renamed `Stmt`→`StmtKind`, type aliases
Grammar (modified)	`src/parser/eta.lalrpop`	Added `@L`/`@R` byte offset captures to all `Expr`/`Stmt` rules
CLI (modified)	`src/cli/cli.rs`	Added `--typecheck` and `--libpath` flags
Main (modified)	`src/main.rs`	Typecheck pipeline, interface loading, arg normalization
S-expression (modified)	`src/sexp.rs`	Updated pattern matching for `Spanned` types
Pretty printer (modified)	`src/pretty.rs`	Updated pattern matching for `Spanned` types

Key interfaces:

```
// Type checker entry points
TypeChecker::new() -> TypeChecker
TypeChecker::register_interface(&mut self, iface: &Interface)
TypeChecker::check_program(&mut self, program: &Program) -> Result<(), SemanticError>

// Internal checking functions
TypeChecker::check_func_def(&mut self, func: &FuncDef) -> Result<(), SemanticError>
TypeChecker::check_expr(&mut self, exp: &Expr) -> Result<SemanticType, SemanticError>
TypeChecker::check_stmt(&mut self, stmt: &Stmt) -> Result<SemanticType, SemanticError>
TypeChecker::check_assign_target(&mut self, target: &AssignTarget, span: Span) ->
TypeResult

// Context operations
Context::new() -> Context
Context::search(&self, name: &String) -> Option<&SemanticType>
Context::push(&mut self, name: &String, typ: &SemanticType) -> bool
Context::enter_scope(&mut self) / Context::exit_scope(&mut self)
```

Data flow (typecheck pipeline):

```
Source File (.eta)
│
▼
read_source_file() ──────────► String (source code)
│
▼
lex_for_parser() ────────────► Vec<(usize, Token, usize)>
│
▼
ProgramParser::parse() ──────► Program AST (with Spanned nodes)
│
▼
load_interfaces() ───────────► Vec<(String, Interface)>  [from .eti files]
│
▼
TypeChecker::register_interface()  [for each interface]
│
▼
TypeChecker::check_program() ► Ok(()) or Err(SemanticError { span, kind })
│
```

```
▼
byte_offset_to_line_col() ──► "Valid Eta Program" or "<line>:<col> error:<msg>"
│
▼
write_output_file() ──────────► .typed file
```

4.2 Core Data Structures

`SemanticType` — the type system's internal representation:

- `Single(Type)` — a single value: `int`, `bool`, or an array type
- `Tuple(Vec<Type>)` — multiple return values from a function
- `Func { args, ret }` — a function signature stored in the context
- `EmptyArray` — the type of `{}`, compatible with any array type
- `Unit` — the type of a statement that does not guarantee a return
- `Void` — the type of a statement that always returns

`SemanticError` — structured error with source location:

```
struct SemanticError { span: Span, kind: SemanticErrorKind }
```

`SemanticErrorKind` is an enum with 13 variants covering scope errors (undeclared identifier, duplicate declaration), type errors (type mismatch, expected array, invalid operator), function errors (not callable, argument count/type mismatch, invalid return type), and control flow errors (break outside loop, missing return).

The `SemanticErrorKind::at(span)` helper provides ergonomic construction: `SemanticErrorKind::TypeMismatch { expected, found }.at(expr.span)`.

`Spanned<T>` — generic AST location wrapper:

```
struct Spanned<T> { node: T, span: Span }
type Expr = Spanned<ExprKind>;
type Stmt = Spanned<StmtKind>;
```

`PartialEq` for `Spanned<T>` compares only the `node` field, ignoring spans. This ensures structural equality checks in the type checker are not affected by source positions.

`ScopedSymbolTable<V>` — scoped symbol table:

Uses a `HashMap<String, Vec<V>>` where each key maps to a stack of values (supporting shadowing). A `log: Vec<LogEntry>` records mutations, with `LogEntry::Marker` entries delimiting scopes. `enter_scope()` pushes a marker; `exit_scope()` pops entries back to the last marker, undoing all bindings from that scope. This gives O(1) amortized scope entry/exit.

4.3 Algorithms

Two-pass program checking:

1. **Pass 1 (signature collection):** Iterate all `TopLevelItems`. For each `FuncDef`, construct a `SemanticType::Func` from its parameter types and return types, and push it into the context. For each `GlobalDecl`, push its type. Duplicate names are rejected.

2. **Pass 2 (body checking):** For each function, save the context, push parameter bindings, set the expected return type, check the body block, then restore the context. For globals with initializers, check the init expression against the declared type.

Expression type checking: Recursive descent over `ExprKind` variants. Binary operators dispatch on the operator to determine valid operand types and result types. Array concatenation (`+` on arrays) requires

matching element types. Equality operators accept any pair of matching types. Function calls verify argument count and types against the context.

Statement type checking: Returns `Unit` or `Void` to track whether a statement guarantees a return. Blocks propagate the type of their last statement (or `Void` if they end in `return`). `if/else` is `Void` only if both branches are `Void`. `while` is always `Unit`.

Assignment checking (4 cases):

1. Procedure call: `Assign([], [FuncCall(...)])` — verify arguments
2. Declaration: `Assign([Decl(name, ty)], [])` — bind name in context
3. Multi-assign from function: `Assign([targets...], [FuncCall(...)])` — match return tuple to targets
4. General n-to-n: `Assign([targets...], [exprs...])` — pairwise type check

4.4 Rust-Specific Design Choices

A significant part of this assignment was choosing how to leverage Rust's type system and idioms to make the type checker correct, concise, and hard to misuse.

`Result<T, E>` and the `?` operator for error propagation. Every checking function returns `Result<SemanticType, SemanticError>` (aliased as `TypeResult`). This eliminated a class of bugs we had in an earlier prototype that accumulated errors into a `Vec<SemanticError>` on the `TypeChecker` struct — it was easy to forget to check the error list, or to continue checking after an error had already invalidated assumptions. With `Result`, the `?` operator makes propagation automatic: `let typ = self.check_expr(expr)?;` either unwraps the successful type or immediately returns the error to the caller. This removed roughly 30 lines of manual error-list management and made every function's control flow visibly correct — if a line uses `?`, you know a failure stops execution there.

Exhaustive `match` for soundness. Rust's compiler enforces that every `match` on an enum is exhaustive. This was critical for the type checker: when we added new `ExprKind` or `StmtKind` variants, the compiler immediately flagged every `match` site that needed updating. For example, `check_expr` matches on all 10 `ExprKind` variants and `check_stmt` on all 4 `StmtKind` variants — if a variant were added to the AST without a corresponding type rule, the code would not compile. Similarly, the `BinOp` match in `check_expr` covers all 14 operators, and adding a new operator forces a handling decision. This is a significant advantage over languages where a missing case would silently fall through or require runtime checks.

Nested pattern matching to avoid code duplication. Rust's pattern matching allowed us to express complex type dispatch concisely. For instance, the `Add` operator must handle five distinct cases (int+int, array+array, array+empty, empty+array, empty+empty), each expressed as a single match arm:

```
match (&typ1, &typ2) {
    (Single(Type::Int), Single(Type::Int)) => Ok(Single(Type::Int)),
    (Single(Type::Array(b1, _)), Single(Type::Array(b2, _))) => { ... }
    (Single(Type::Array(_, _)), EmptyArray) => Ok(typ1.clone()),
    (EmptyArray, Single(Type::Array(_, _))) => Ok(typ2.clone()),
    (EmptyArray, EmptyArray) => Ok(EmptyArray),
    _ => Err(InvalidBinaryOp { ... }.at(span)),
}
```

The `_` wildcard catch-all guarantees all other combinations produce an error. Without pattern matching, this would require nested `if/else` chains or separate functions for each case.

Generic `Spanned<T>` with type aliases. We considered two alternatives for carrying source locations: (a) adding `span: Span` fields to `ExprKind` and `StmtKind` directly, or (b) wrapping in a generic struct. Option (a) would have cluttered every pattern match — every arm would need to bind or ignore the span field. With option (b), pattern matching on the inner node is a single `.node` access: `match &expr.node { ExprKind::Var(v) => ... }`. The type aliases `type Expr = Spanned<ExprKind>` let function signatures stay clean while carrying location data. The custom `PartialEq` implementation (comparing only `.node`, ignoring `.span`) meant existing equality checks in the type checker worked without modification.

`SemanticErrorKind::at(span)` builder pattern. Constructing a `SemanticError` requires both a `kind` and a `span`. Rather than writing `SemanticError { span: expr.span, kind: SemanticErrorKind::TypeMismatch { ... } }` everywhere, the `at()` method on `SemanticErrorKind` lets us write the kind first and attach the span last: `SemanticErrorKind::TypeMismatch { expected, found }.at(expr.span)`. This reads naturally (describe the error, then say where it is) and avoids repetitive struct construction.

Generics in `ScopedSymbolTable<V>`. The symbol table is generic over its value type `V`, not hardcoded to `SemanticType`. This made unit testing straightforward — tests use `ScopedSymbolTable<i32>` to verify scope mechanics with simple integers, without needing to construct `SemanticType` values. The `Context` wrapper specializes it to `ScopedSymbolTable<SemanticType>` for the actual type checker.

4.5 Implementation Strategy and Collaboration

We used a bottom-up approach, building foundational components first:

1. Designed and implemented the scoped symbol table and context
2. Built the core type checker logic (expressions, statements, functions)
3. Added span propagation to the AST and grammar
4. Integrated error handling with `Result` types and structured `SemanticError`
5. Wired up the CLI `--typecheck` flag, interface loading, and output pipeline
6. End-to-end testing with the course test harness

Challenges:

- **AST span propagation:** Adding `Spanned<T>` wrappers required coordinated changes across `ast.rs`, `eta.lalrpop`, `sexp.rs`, `pretty.rs`, `check.rs`, and all test files. The `ExprKind`/`StmtKind` rename and type alias approach minimized churn in pattern matching code.

- **Result type refactoring:** Converting the type checker from error accumulation to fail-fast `Result` propagation required rewriting every checking function to use `Ok(...)`, `Err(...)`, and `?` consistently.

- **Single-dash CLI flag compatibility:** The course test harness passes `-libpath` (single dash) which `clap` interprets as chained short flags `-l -i -b ...`. We solved this with a preprocessing step that normalizes single-dash long arguments before clap parsing.

Code reuse: PA1 lexer code was unchanged. PA2 parser and S-expression code required only mechanical updates for `Spanned` types. The CLI module was extended with new flags.

Who wrote what:

Team Member	Responsibilities
Charles	AST span propagation (`Span`, `Spanned<T>`), error handling refactoring (`Result` types, `SemanticError`/`SemanticErrorKind` design), `byte_offset_to_line_col` integration
Temi	CLI functionality (`--typecheck` flag, `--libpath`, `-D` integration), typecheck output pipeline, arg normalization
Yoseph	Core type checker logic (`check_expr`, `check_stmt`, `check_func_def`, `check_program`), assignment checking (4 cases), context stub
Fadi	Scoped symbol table design and implementation (`ScopedSymbolTable`, `Context`), context unit tests

---

5. Testing

5.1 Test Plan

We designed tests at two levels: unit tests for individual components and integration tests via the course test harness.

Unit tests (run via `cargo test`):

- **Context tests (8 tests):** Basic shadowing, multiple puts in same scope, independent keys, deep nesting (3 levels), exit scope with no bindings, scope debug helpers (depth/len), panic on exit without marker. These verify the correctness of the scoped symbol table's enter/exit/put/get operations.
- **S-expression tests (24 tests):** Carried forward from PA2, updated for `Spanned` types. Verify AST-to-S-expression conversion for all node types.
- **Pretty printer tests (12 tests):** Carried forward from PA2, updated for `Spanned` types.

Integration tests (run via `eth` test harness):

The course-provided PA3 test suite includes 29 typecheck tests covering: valid programs with functions, globals, arrays, multi-dimensional arrays, multi-return functions, multi-assignment, forward references, library usage (`use io`), and programs with intentional type errors.

5.2 Results

- **Unit tests:** All 44 tests pass (`cargo test`): 8 context, 24 S-expression, 12 pretty printer.
- **Integration tests:** 29 out of 29 typecheck tests pass at time of writing.
- **Regression:** PA1 lexer and PA2 parser tests pass when run from their respective test directories.
- **Systems tested:** macOS (Darwin 25.2.0) for development, Linux (course Docker image `sileiren/cs4120-env:v1`) for harness testing.

---

6. Work Plan

The work was divided by component with clear dependency ordering:

1. **Fadi:** Designed and implemented `ScopedSymbolTable` and `Context` with unit tests (independent, no dependencies).
2. **Yoseph:** Built core type checker logic using a context stub, then integrated with Fadi's real context implementation.
3. **Charles:** Added span propagation to AST and grammar, refactored error handling to use `Result` types, structured `SemanticError` with spans.
4. **Temi:** Implemented CLI `--typecheck` pipeline, interface file loading, output formatting, and arg normalization.

Integration happened in around three merges: The bulk of the type checker work was developed by Yoseph in his own branch. Charles picked up off of Yoseph's branch and added proper error handling with spans. Fadi's type context and Temi's CLI work was developed mostly in parallel and integrated last.

---

7. AI Usage

We used AI tools (Claude, GitHub Copilot) in the following ways:

- **Error handling refactoring:** AI helped identify inconsistencies in `Result` type usage across the type checker and suggested the `?` operator pattern for fail-fast propagation.
- **Span propagation design:** AI helped evaluate the tradeoff between adding span fields to each AST node vs. a generic `Spanned<T>` wrapper, and assisted with the mechanical updates across grammar

rules and pattern matching code.
- **CLI debugging:** AI diagnosed the `-libpath` single-dash issue by tracing how `clap` interprets chained short flags.
- **Boilerplate:** Copilot provided autocomplete for repetitive match arms in the type checker.

AI was helpful for accelerating mechanical refactoring and diagnosing non-obvious issues (like the CLI flag parsing). Core type checking logic and the scoped symbol table were designed and implemented by team members directly.

---

8. Known Problems

- The type checker uses `context.clone()` to save/restore state around scoped constructs (blocks, if/else, while) rather than using the context's `enter_scope`/`exit_scope` methods. This is correct but less efficient for deeply nested programs.
- Some `SemanticErrorKind` variants (`BreakOutsideLoop`) are defined but not yet used, as loop break statements are not yet fully implemented.

---

9. Comments

The biggest surprise was the `-libpath` single-dash issue — the test harness was passing Java-style flags that clap silently misinterpreted as chained short flags. This took time to diagnose because the error message (`unexpected argument '-b' found`) gave no indication of the real problem.

The `Spanned<T>` wrapper approach worked well in practice. The main cost was the one-time effort of updating all grammar rules and pattern matching sites, but once done, every expression and statement carried its source location with zero ongoing effort.